📥 **Download as PDF:**

Press `Ctrl+P` (or `Cmd+P` on Mac), then select "Save as PDF"

🖨️ Open Print Dialog

AI COACH

Pose Estimation Platform Architecture

Modern, Real-time AI-Powered Fitness Coaching with Live Pose Detection

Complete Architecture & Implementation Guide

Free & Open-Source Tools Only

Generated: 2024

📋 Table of Contents

1. Project Overview
2. System Architecture
3. Technology Stack
4. Component Breakdown
5. Data Flow & Real-Time Example
6. Performance Targets
7. UI/UX Design Specs
8. Exercise Database
9. Implementation Phases
10. Deployment & Checklist

11. File Structure

1. Project Overview

Build a modern, real-time AI-powered fitness coaching platform that combines:

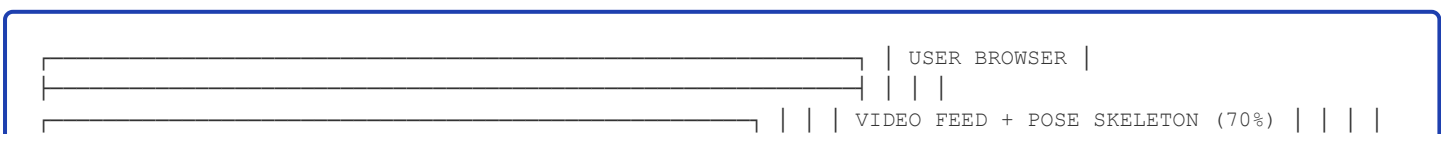
- **Real-Time Detection** Live pose estimation via webcam (<100ms latency)
- **AI Analysis** Real-time form scoring & automatic feedback
- **Smart Corrections** AI-generated correction images & natural voice guidance
- **Progress Tracking** Session analytics, form improvements, achievements
- **100% Free** All open-source tools, no paid APIs required

Key Differentiators from Reference (Personal Hour App)

Feature	Reference App	AI Coach (Ours)
Pose Detection	Basic green skeleton	Color-coded joints, animated, real-time scoring
UI/UX	Basic iPad interface	Modern dark theme, smooth animations, responsive
Corrections	Text feedback only	Text + AI-generated images + voice guidance
Form Scoring	Simple binary (correct/incorrect)	Detailed percentage breakdown by body part
Voice	Pre-recorded audio	Real-time AI-generated voice guidance
Cost	Paid subscription	Completely free, self-hosted

2. System Architecture

High-Level Overview



```
├─ Webcam (30-60 FPS) │ │ │ │ ├─ MediaPipe.js (5-15ms detection) │ │ │ │ ├─ Skeleton Overlay (animated, color-coded) │ │ │ │ └─ AI Coach Avatar + Voice Transcript │ │ │
└──────────────────────────────────────────────────────┘ │ │
┌──────────────────────────────────────────────────────┐ │ │ DASHBOARD (30%) │ │ │ │ ├─ Form Score: 89/100 │ │ │ │ ├─ Rep Counter: 12/20 │ │ │ │ ├─ Current Issue + AI Image │ │ │ │ ├─ Voice Guidance Transcript │ │ │ │ └─ Session Metrics │ │ │ └──────────────────────────────────────────────────────┘ │ │
│ │ ↓ WebSocket (non-blocking) │ └──────────────────────────────────────────────────────┘ │ ▼
┌──────────────────────────────────────┐ │ BACKEND (FastAPI/Node.js) │
├──────────────────────────────────────┤ │ │ │ ┌─ Pose Validator (sync, <5ms) │ │ ├─ LLM: Correction Text (async) │ │ ├─ Image Gen: Illustrations (async) │ │ ├─ TTS: Voice Synthesis (async) │ │ └─ Database: PostgreSQL │ │ │ └──────────────────────────────────────┘
```

Component Interaction Flow

```
User Starts Workout
    ↓
Frontend loads exercise details
    ↓
MediaPipe.js captures webcam & detects 33 keypoints (browser-side)
    ↓
Local validation compares angles to ideal (frontend, <5ms)
    ↓
Form score calculated & displayed in real-time (green/yellow/red)
    ↓
Send keypoints to backend via WebSocket (async, non-blocking)
    ↓
Backend queues correction generation:
    • LLM generates correction text (5-10 sec)
    • Stable Diffusion generates image (2-30 sec)
    • Pyttsx3/Google TTS generates voice (5-20 sec)
    ↓
Updates stream back to frontend via WebSocket
    ↓
User sees feedback immediately (pose visual)
User reads correction text after 5-10 sec
User hears voice guidance after ~15 sec
```

3. Technology Stack

Frontend

Core

- **Framework:** React 18 + Next.js 14 (TypeScript)
- **Styling:** Tailwind CSS + Framer Motion (animations)
- **UI Components:** Shadcn/ui (accessible component library)

- **UI Components:** Shadcn/ui (accessible component library)
- **3D/Visualization:** Three.js or Babylon.js (skeleton rendering)
- **Pose Detection:** MediaPipe.js (WASM, in-browser)

Real-Time Communication

- **WebSocket:** Socket.io-client (persistent connection)
- **State Management:** Zustand or Jotai (lightweight)

Forms & Utils

- **Forms:** React Hook Form
- **HTTP:** Axios or Fetch API

Backend

Framework & Server

- **Language:** Python 3.10+
- **Framework:** FastAPI (async, WebSocket support)
- **Alternative:** Node.js + Express + ws

AI & LLM

- **Pose Validation:** Custom Python logic (angle comparison)
- **LLM Service:** Ollama (self-hosted)
- **LLM Model:** Mistral 7B or Llama 2 7B
- **Image Generation:** Stable Diffusion (local)
- **Text-to-Speech:** Pyttsx3 (local) or Google Cloud TTS (free 4M chars/mo)

Database & Async

- **Database:** PostgreSQL + Prisma ORM
- **Task Queue:** Celery + Redis (optional, for heavy async tasks)

Deployment Infrastructure

Hosting

- **Frontend:** Vercel (free tier, edge functions)

- **Backend API:** Railway or Render (free tier)
- **Database:** Railway PostgreSQL (free tier)
- **LLM Service:** Same server as backend or dedicated cheap VPS ($5-10/mo)

Alternative: Self-Hosted (Recommended for Performance)

- **Single VPS:** DigitalOcean/Linode ($5-10/mo)
- **All services:** Frontend, Backend, PostgreSQL, Ollama, Stable Diffusion on one machine
- **Benefit:** WebSocket latency under 5ms (vs 50-100ms to cloud)

4. Core Components Breakdown

Frontend Components

VideoPoseContainer.tsx

- Manages webcam stream capture via getUserMedia()
- Loads MediaPipe pose detection (WASM module)
- Passes keypoints to skeleton overlay at 30-60 FPS
- Sends pose data to backend via WebSocket (only when issues detected)

PoseSkeleton.tsx

- Renders 33 MediaPipe keypoints as animated skeleton
- Color-coded joints:
  - Green = Good form
  - Yellow = Minor correction needed
  - Red = Major issue
- Smooth line interpolation between joints
- Pulsing animation on joints needing correction

FormScoreMeter.tsx

- Circular progress indicator (0-100%)
- Real-time percentage display (updates every frame)
- Breakdown by body part:
  - Upper body alignment (%)
  - Core engagement (%)
  - Lower body alignment (%)

    - Range of motion (%)

- Smooth counter animation when score changes

CurrentIssueCard.tsx

- Displays current form issue in plain text
- AI-generated correction image (arrives async)
- Actionable "How to fix" tips
- Slide-in animation when new issue detected
- Auto-disappears when issue is corrected

AICoachOverlay.tsx

- Animated AI coach avatar (or simple avatar image)
- Real-time voice transcript display
- Play/pause audio controls
- Volume slider
- Waveform animation during speech

DashboardPanel.tsx

- Container for all right-side widgets (30% of screen)
- Sticky positioning during scroll
- Collapsible on mobile → Bottom sheet

ExerciseNavigator.tsx

- Carousel of exercises in current workout
- Jump to specific exercise mid-workout
- Preview upcoming exercise
- Track current position in workout

SessionSummary.tsx

- Duration, estimated calories burned
- Average form score across all exercises
- Most common form issues
- Form improvement vs. previous session
- Workout streak counter 🔥
- Badges earned in this session

Backend Services

pose_validator.py

Purpose: Validates detected pose against ideal angles for exercise

```
def validate_pose(keypoints, exercise_type) -> Feedback:
    angles = compute_joint_angles(keypoints)
    ideal_angles = EXERCISE_DATABASE[exercise_type]

    score = 100
    issues = []

    for joint, detected_angle in angles.items():
        ideal_range = ideal_angles[joint]
        if not in_range(detected_angle, ideal_range):
            deviation = abs(detected_angle - ideal_range.center)
            severity = min(100, deviation / 30)

            issues.append({
                "joint": joint,
                "detected": detected_angle,
                "ideal": ideal_range,
                "severity": severity
            })
            score -= severity * 0.5

    return Feedback(score=max(0, score), issues=issues)
```

correction_generator.py

Purpose: Generates natural language corrections using LLM

```
async def generate_correction(feedback: Feedback) -> str:
    prompt = f"""
    The user is doing a {feedback.exercise_type}.

    Issues detected:
    - {feedback.issues[0].joint}: angle {feedback.issues[0].detected}°
      (should be {feedback.issues[0].ideal})

    Provide a brief, actionable correction (1-2 sentences).
    Use encouraging, coaching tone.
    """

    correction = await ollama.generate(
        model="mistral",
        prompt=prompt
    )
    return correction
```

image_generator.py

Purpose: Generates correction images using Stable Diffusion

```
async def generate_correction_image(feedback: Feedback) -> str:
    prompt = f"""
    Professional fitness photo showing correct {feedback.exercise_type} form.
    Emphasize: {feedback.primary_issue}
    Side view, white background, clear alignment markers.
    """

    image = await stable_diffusion.generate(
        prompt=prompt,
        steps=30,
        guidance_scale=7.5
    )

    image_url = save_and_upload_image(image)
    return image_url
```

tts_service.py

Purpose: Converts correction text to voice guidance

```
async def text_to_speech(text: str) -> bytes:
    # Option 1: Pyttsx3 (local, instant, free)
    engine = pyttsx3.init()
    engine.say(text)
    audio_bytes = engine.save()

    # Option 2: Google Cloud TTS (better quality, free 4M chars/mo)
    # client = texttospeech.TextToSpeechClient()
    # response = client.synthesize_speech(...)

    return audio_bytes
```

websocket_handler.py

Purpose: Maintains persistent connection during workout, broadcasts real-time feedback

```
@app.websocket("/ws/workout/{user_id}")
async def workout_websocket(websocket: WebSocket, user_id: str):
    await websocket.accept()

    while True:
        # Receive pose keypoints from frontend
        data = await websocket.receive_json()
        feedback = validate_pose(data['keypoints'], data['exercise'])

        # Send form score immediately (no wait)
        await websocket.send_json({
            "type": "score_update",
            "score": feedback.score
```

```
        })

        # Queue async tasks (don't block)
        if feedback.has_new_issues:
            asyncio.create_task(
                generate_and_broadcast_correction(websocket, feedback)
            )
            asyncio.create_task(
                generate_and_broadcast_image(websocket, feedback)
            )
            asyncio.create_task(
                generate_and_broadcast_voice(websocket, feedback)
            )
```

5. Data Flow & Real-Time Example

Timeline: User Does Jumping Jacks

Time (ms)	Action	Component	Result
0	Webcam captures frame	VideoPoseContainer	Raw video frame
5	MediaPipe detects 33 keypoints	PoseSkeleton	Skeleton keypoints
10	Compute joint angles locally	Frontend (sync)	Angle data
15	Compare to ideal angles	Frontend (sync)	Form score: 92%
20	Draw skeleton with colors	PoseSkeleton	Green skeleton overlay
30	**Total frontend latency**	**User sees feedback in 30ms ✓**	
30	Send keypoints to backend (async)	WebSocket	Non-blocking
35	Backend validates pose	pose_validator	New issue detected: elbows too wide
40	Send score update to frontend	WebSocket	Score: 87% (yellow elbows)
50	LLM generates correction text	correction_generator	"Keep elbows closer to body"
5000	Send correction text to	WebSocket	Display in CoachingPanel

5000	frontend	WebSocket	Display in CurrentIssueCard
50	Stable Diffusion starts generating image	image_generator	Creating correction visual...
10000	Image generation completes	image_generator	Correction image ready
10100	Send image URL to frontend	WebSocket	Display in CurrentIssueCard
50	TTS synthesizes voice guidance	tts_service	Converting text to speech...
15000	Voice synthesis completes	tts_service	Audio ready
15100	Stream audio to frontend + play	WebSocket + AICoachOverlay	User hears guidance

Key Insight: Non-Blocking Architecture

> ✓ **User Experience:**
>
> - Sees pose feedback instantly (30ms) → No perceived lag
> - Reads correction text after 5-10 seconds
> - Sees correction image after 10-30 seconds
> - Hears voice guidance after ~15 seconds
> - **At no point does the UI freeze or wait for backend**

6. Performance Targets

Metric	Target	Actual (Optimized)	Notes
Pose Detection Latency	<100ms	10-30ms	MediaPipe WASM in browser
Form Score Update	<50ms	30-40ms	Local angle comparison
Visual Feedback	Instant	0-5ms	Canvas draw, no network
Correction Text	5-10s	5-10s	Ollama LLM inference
Correction Image	2-30s	2-30s	Stable Diffusion (CPU: slower)
Voice Guidance	5-20s	5-20s	TTS synthesis

🚀 **Pose detection never blocks UI; corrections queue asynchronously**

⚠️ **Performance Notes:**

- CPU-bound tasks (Stable Diffusion, LLM) run in background
- GPU acceleration recommended for image generation (free tier unlikely has it)
- Cache common corrections to reduce generation time
- Pre-generate illustrations for popular exercises

7. UI/UX Design Specifications

Color Palette (Dark Modern Theme)

Element	Color	Usage
Background	#0f1419	Page background (very dark)
Surface	#1a1f2e	Cards, panels, overlays
Primary (Good)	#00d9ff	Good form, success states, active elements
Warning	#ffaa00	Minor corrections needed
Danger	#ff0055	Major form issues
Success	#00ff88	Exercise completed
Text Primary	#e0e0e0	Main text
Text Secondary	#a0a0a0	Secondary text, hints

Typography

- **Headings:** Inter Bold, 24-48px
- **Body Text:** Inter Regular, 14-16px
- **Metrics/Numbers:** JetBrains Mono, 24-32px (form score, reps)
- **Labels:** Poppins Medium, 12-14px

Layout Responsive Design

Breakpoint	Layout	Video	Dashboard
Desktop (1024px+)	Side-by-side	70% width	30% width, sticky
Tablet (768px-1023px)	Full video, collapsible dashboard	100% width	Slide-in from right
Mobile (< 768px)	Full video, bottom sheet dashboard	100% width	Bottom drawer (drag-up)

Animation & Micro-Interactions

- **Score change:** Counter animate (1s with easing)
- **Skeleton color update:** Smooth transition (200ms)
- **Issue detection:** Pulsing animation on affected joints (infinite until fixed)
- **Card appear:** Slide-in from right (300ms, ease-out)
- **Issue disappear:** Fade-out (200ms)
- **Voice playback:** Animated waveform or speaking avatar
- **Skeleton drawing:** Lines animate in sequence (500ms total)

8. Exercise Database Structure

Store exercise metadata and ideal pose angles in JSON format:

```
{
  "jumping_jack": {
    "name": "Jumping Jack",
    "category": "cardio",
    "difficulty": "beginner",
    "duration_sec": 30,
    "rep_target": 20,
    "description": "Full-body cardio exercise...",

    "ideal_angles": {
      "left_shoulder": [160, 180],
      "right_shoulder": [160, 180],
      "left_elbow": [160, 180],
      "right_elbow": [160, 180],
      "left_hip": [40, 70],
      "right_hip": [40, 70],
      "left_knee": [160, 180],
      "right_knee": [160, 180]
    },

    "common_issues": [
      "elbows_too_wide",
      "knees not straight",
```

```
      "shoulders_hunched"
    ],

    "correction_tips": [
      "Keep elbows close to body for control",
      "Engage your core throughout",
      "Land softly on balls of feet",
      "Don't let knees cave inward"
    ],

    "voice_intro": "Let's do jumping jacks! Remember to keep your
                    elbows close to your body. I'll guide you on
                    your form.",

    "voice_encouragement": [
      "You're doing great!",
      "Keep up the pace!",
      "Nice form!"
    ]
  }
}
```

9. Implementation Phases

Phase 1: MVP Frontend (Weeks 1-2)

✓ Must Have

- React + Next.js project setup
- Tailwind CSS + dark theme
- MediaPipe.js integration (webcam → pose detection)
- PoseSkeleton component (smooth animation, color-coded)
- FormScoreMeter (real-time percentage)
- Rep counter with auto-increment
- Basic dashboard layout
- Exercise selection screen

Phase 2: Backend Integration (Weeks 3-4)

✓ Must Have

- FastAPI backend setup
- WebSocket endpoint for real-time updates
- Pose validation logic (angle comparison)
- PostgreSQL database + schema
- Ollama integration (Mistral 7B)
- Correction text generation
- Pyttsx3 TTS integration
- Session storage (user progress)

Phase 3: AI Features (Weeks 5-6)

> ✓ Must Have

- Stable Diffusion for correction images
- Image caching/storage
- Exercise library (20+ exercises with angles)
- Better prompt engineering for LLM
- Error handling & fallbacks
- Async task queue (Celery optional)

Phase 4: Polish & UX (Weeks 7-8)

> ✓ Nice to Have

- Smooth animations & transitions
- Session summary dashboard
- Progress tracking graphs
- User authentication
- Mobile responsiveness testing
- Performance optimization
- Deployment to production

Phase 5: Advanced Features (Optional)

> ✓ Advanced

- Gamification (badges, streaks, leaderboards)
- Custom workout builder

- Social features (share progress, challenges)
- Mobile app (React Native or Flutter)
- Wearable integration (heart rate, etc.)
- Advanced analytics & form tracking

10. Deployment & Checklist

Local Development Setup

```
# Terminal 1: Frontend
cd frontend
npm install
npm run dev
# → http://localhost:3000

# Terminal 2: Backend
cd backend
pip install -r requirements.txt
python -m uvicorn main:app --reload
# → http://localhost:8000

# Terminal 3: Ollama (LLM Service)
ollama serve
# → http://localhost:11434

# Terminal 4: Database
# Docker or local PostgreSQL
docker run -d -p 5432:5432 \
  -e POSTGRES_PASSWORD=password \
  postgres:15
```

Production Deployment Options

Option A: Cloud Deployment (Recommended for Learning)

- **Frontend:** Push to GitHub → Auto-deploy to Vercel
- **Backend:** Push to Railway/Render → Auto-deploy
- **Database:** Railway PostgreSQL (free tier)
- **LLM:** Separate Railway dyno with Ollama installed

Option B: Self-Hosted (Recommended for Performance)

- **VPS:** DigitalOcean / Linode / Hetzner ($5-10/mo)

- **All services on one machine:** Frontend, Backend, PostgreSQL, Ollama
- **WebSocket latency:** <5ms (vs 50-100ms to cloud)
- **Setup:** Docker Compose with all services

Environment Variables

```
# Backend (.env)
DATABASE_URL=postgresql://user:pass@localhost/ai_coach
OLLAMA_API_URL=http://localhost:11434
CORS_ORIGINS=http://localhost:3000,https://yourdomain.com
SECRET_KEY=your-secret-key-here
ENVIRONMENT=development

# Frontend (.env.local)
NEXT_PUBLIC_BACKEND_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000/ws
```

Production Checklist

Item	Status	Notes
Frontend deployed to Vercel	☐	GitHub Actions auto-deploy on push
Backend deployed (Railway/Render)	☐	Environment variables configured
PostgreSQL database running	☐	Backups configured
Ollama service accessible	☐	LLM models downloaded
Stable Diffusion configured	☐	Models cached locally
WebSocket connections tested	☐	Real-time updates working
SSL/HTTPS configured	☐	Vercel auto-handles; backend needs cert
Error logging & monitoring	☐	Sentry or similar
Performance tested	☐	Pose detection < 100ms latency
Load testing completed	☐	Expected concurrent users: ?
Mobile testing completed	☐	iOS Safari, Android Chrome
User documentation written	☐	Setup guide, FAQ

11. File Structure & Architecture

```
ai-coach/
│
├── frontend/ (Next.js + React)
│   ├── src/
│   │   ├── components/
│   │   │   ├── VideoPoseContainer.tsx
│   │   │   ├── PoseSkeleton.tsx
│   │   │   ├── FormScoreMeter.tsx
│   │   │   ├── CurrentIssueCard.tsx
│   │   │   ├── AICoachOverlay.tsx
│   │   │   ├── DashboardPanel.tsx
│   │   │   ├── ExerciseNavigator.tsx
│   │   │   ├── SessionSummary.tsx
│   │   │   ├── ExerciseSelector.tsx
│   │   │   └── Layout.tsx
│   │   │
│   │   ├── hooks/
│   │   │   ├── usePoseDetection.ts
│   │   │   ├── useWebSocket.ts
│   │   │   ├── useWorkoutState.ts
│   │   │   └── useLocalStorage.ts
│   │   │
│   │   ├── pages/
│   │   │   ├── index.tsx (home)
│   │   │   ├── workout/[id].tsx (live workout)
│   │   │   ├── dashboard.tsx (analytics)
│   │   │   ├── history.tsx (past sessions)
│   │   │   └── settings.tsx
│   │   │
│   │   ├── styles/
│   │   │   ├── globals.css (Tailwind)
│   │   │   └── animations.css
│   │   │
│   │   ├── utils/
│   │   │   ├── api.ts (HTTP client)
│   │   │   ├── websocket.ts (WebSocket client)
│   │   │   ├── poseValidation.ts (angle logic)
│   │   │   └── formatters.ts (display helpers)
│   │   │
│   │   ├── types/
│   │   │   ├── index.ts (TypeScript interfaces)
│   │   │   └── mediapipe.ts
│   │   │
│   │   └── App.tsx (root component)
│   │
│   ├── public/
│   │   ├── exercises/ (exercise images)
│   │   └── avatars/ (AI coach avatar)
│   │
```

```
│   ├── package.json
│   ├── tsconfig.json
│   ├── tailwind.config.js
│   └── next.config.js
│
├── backend/ (FastAPI + Python)
│   ├── main.py (FastAPI app + WebSocket)
│   ├── models.py (Pydantic schemas)
│   ├── database.py (PostgreSQL setup)
│   │
│   ├── services/
│   │   ├── pose_validator.py (angle comparison)
│   │   ├── correction_generator.py (LLM)
│   │   ├── image_generator.py (Stable Diffusion)
│   │   ├── tts_service.py (voice synthesis)
│   │   └── user_service.py (database ops)
│   │
│   ├── routes/
│   │   ├── auth.py (user login/signup)
│   │   ├── workouts.py (workout CRUD)
│   │   ├── exercises.py (exercise library)
│   │   └── analytics.py (user stats)
│   │
│   ├── config.py (environment variables)
│   ├── requirements.txt
│   ├── docker-compose.yml
│   └── Dockerfile
│
├── exercises/
│   └── exercises.json (exercise database with angles)
│
├── docs/
│   ├── API.md (API documentation)
│   ├── SETUP.md (installation guide)
│   ├── ARCHITECTURE.md (this document)
│   └── TROUBLESHOOTING.md
│
├── .github/
│   └── workflows/
│       ├── frontend-deploy.yml (Vercel auto-deploy)
│       └── backend-deploy.yml (Railway/Render auto-deploy)
│
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

12. Quick Start Commands

Clone & Setup

```
# Clone repository
git clone <your-repo>
cd ai-coach

# Frontend setup
cd frontend
npm install
cp .env.example .env.local

# Backend setup
cd ../backend
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env

# Download Ollama models
ollama pull mistral
ollama pull stable-diffusion
```

Run Locally

```
# Terminal 1: Ollama
ollama serve

# Terminal 2: PostgreSQL
docker run -d -p 5432:5432 \
  -e POSTGRES_PASSWORD=password \
  --name ai-coach-db postgres:15

# Terminal 3: Backend
cd backend
python -m uvicorn main:app --reload

# Terminal 4: Frontend
cd frontend
npm run dev
```

Deploy to Production

```
# Frontend (Vercel)
git push origin main
# → Auto-deploys to Vercel

# Backend (Railway)
```

```
railway link
railway up
# → Auto-deploys to Railway

# Set environment variables on Railway dashboard
```

AI Coach - Pose Estimation Platform Architecture Document